

Progress Report of *RLHF for Open-Ended Instruction Following*

Yehong Qiu*

1 Project Overview

The project investigates reinforcement learning from human feedback (RLHF) methods for instruction following language models. During the first month of the project, I focused primarily on implementing the required RLHF algorithms from the Berkeley CS185/285 26Spring project framework. All implementation tasks in the provided codebase have been completed, and the training pipeline is operational. I have begun running DPO experiments and collecting diagnostic metrics to better understand optimization behavior. While the experimental phase is still ongoing and definitive performance conclusions are premature, the current results indicate that the framework is functioning as intended and is ready for systematic evaluation and further investigation.

2 Progress to Date

Here is a brief write-up for the current progress:

- Set up the complete training environment and evaluation workflow.
- Studied the theoretical foundations of reward modeling, DPO, IPO, AOT, GRPO, DrGRPO, and GSPO.
- Completed all required TODO implementations in the starter codebase.
- Conducted sanity checks through training runs and diagnostic monitoring, providing evidence that the DPO and reward-model implementations behave consistently with theoretical expectations.
- Trained initial reward-model and DPO policy baselines.
- Began DPO hyperparameter tuning experiments.
- Generated preliminary diagnostic plots for DPO training behavior analysis.

3 Preliminary Observations

Here is presented the current experimental results on the DPO algorithm. The reference corrected margin is

$$\Delta_{\theta}(x, y^+, y^-) = \log\left(\frac{\pi_{\theta}(y^+|x)}{\pi_{\theta}(y^-|x)}\right) - \log\left(\frac{\pi_{\text{ref}}(y^+|x)}{\pi_{\text{ref}}(y^-|x)}\right)$$

in which x is a prompt, y is a full response sequence, y^+ and y^- denote the preferred and rejected responses, π_{θ} is the current policy/LM model, and π_{ref} is the frozen reference policy/base model. The preference margin quantifies by how much amount the current policy bias more towards the preferred response to rejected response than the fixed reference policy.

A benign reference corrected margin should be positive, so we define the reference corrected accuracy as the percentage of samples with positive reference corrected margin.

*yehqiu@ucdavis.edu

The policy margin is essentially the first term in the reference corrected margin:

$$\text{policy margin} = \log\left(\frac{\pi_{\theta}(y^+|x)}{\pi_{\theta}(y^-|x)}\right)$$

which should also be positive in the benign case.

The training objective for the DPO algorithm is to minimize the loss:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log \sigma(\beta \Delta_{\theta}(x, y^+, y^-))]$$

in which $\sigma(\cdot)$ is the logistic function and $\beta > 0$ is the preference-loss scale parameter.

Fig. 1 presents the changing patterns of the 4 aforementioned in-sample diagnostics throughout the training process, and fig. 2 shows the out-of-sample diagnostics. The diagnostics exhibit trends that are qualitatively consistent with successful DPO optimization, except that the in-sample metrics are noisier due to the innate fluctuation of token probabilities across different instances.

As for the current choices on hyperparameters(β , the learning rate, the effective training batch size, and the total training steps, please see Appendix A).

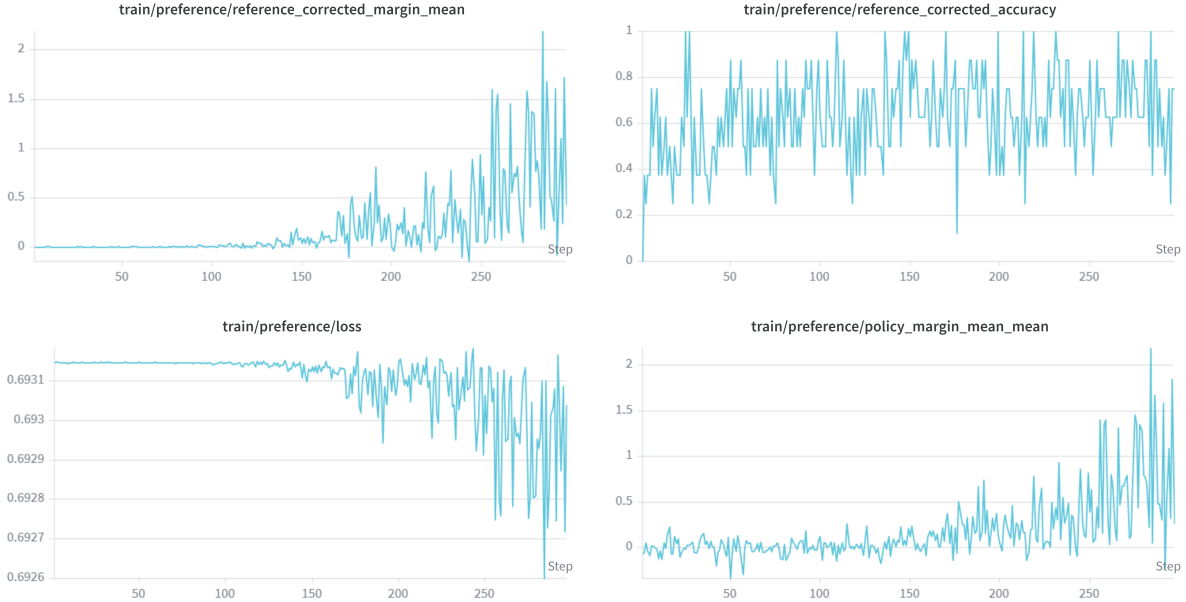


Figure 1: Diagnostic metrics on the training set for the DPO algorithm. Despite the fluctuation throughout the training process, the general trend is correct: the reference corrected margin averaged over the prompt-response pairs is positive and increasing; the reference corrected accuracy goes up slowly; the train loss decreases gently; and the mean policy margin averaged over all data instances is positive and increasing, too.

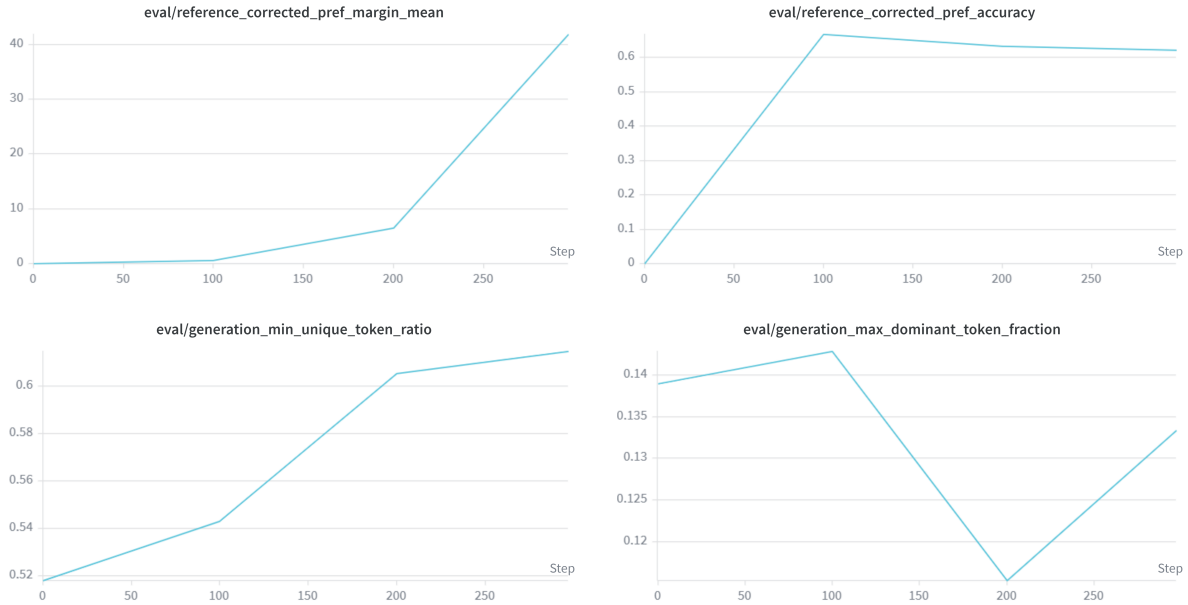


Figure 2: Diagnostic metrics on the held-out evaluation set for the DPO algorithm. The metrics reflect that training under the current setting is healthy but underperformed: The reference corrected margin averaged over all held-out instances is steadily increasing; the reference corrected accuracy goes up to 0.6 and then plateaus, indicating a slightly underperformed performance; the minimum unique token ratio stays large (>0.6) while the maximum dominant token fraction is small (~ 0.1), suggesting the generated response from the trained policy does not degrade into repetition, and that there is no obvious pathologic generation pattern.

The actual win-rate of the current DPO policy against the fixed base model is 0.3051 over the 128 public evaluation prompts, which do not appear in both the training and the held-out evaluation set. The current win-rate is fair but still below the required 0.51 threshold, meaning that the current policy underperforms the target benchmark and requires further investigation. Below is an example generated by the learnt policy given the user prompt, which already shows a fair capability at instruction following, but notice the tricky logic in the prompt — the current policy cannot understand the subtle and unusual logic hidden within the question: What the user actually wants to ask is "Why not all of the fox news is fake?" In other words, they imply that "Some fox news is true." But the policy returns an essay essentially on "Why some fox news is false."

Prompt Why isn't all of the fox news not real

Response Fox News and other cable news outlets often reflect diverse perspectives on various issues, leveraging the power of media as a mirror to societal concerns, but also amplifying certain narratives through selective framing and audience appeal, which can create echo chambers for specific viewpoints:

1. **Selective Presentation** - Fox News, for instance, might highlight stories that align with its conservative agenda while downplaying or ignoring more nuanced or opposing views.
2. **Audience Appeal** - Understanding audience preferences is crucial. Fox's content strategy often targets viewers who share similar political leanings, reinforcing

existing beliefs rather than challenging them comprehensively.

3. **Echo Chambers** - Media environments tend to form "echo chambers" where people only consume information that confirms their pre-existing beliefs, leading to a skewed perception of reality.

4. **Bias in Reporting** - Even reputable journalism has biases. What gets reported and how it's reported can significantly influence what audiences perceive as "real."

5. **Engagement Over Truth** - Engagement metrics (likes, shares) sometimes override journalistic integrity, pushing stories that generate high viewer interaction over those that require deeper investigative reporting.

Thus, while not all aspects of Fox News are inherently false, they often operate within frameworks that emphasize authenticity through selective presentation and engagement-driven storytelling, making it challenging...[truncated]

4 Current Challenges

The current challenge lies in selecting the stable DPO hyperparameters so that the algorithm can beat the performance threshold. There are engineering issues such as avoiding numerical overflow or underflow. For example, one technical issue I tackled is to rewrite the DPO loss function to a mathematically equivalence in order to guarantee computation stability.(For details, see Appendix B.)

Another technical challenge for taming the DPO algorithm is that even when the internal metrics behave normally, the final win-rate might not performs well. Besides, the algorithm is prone to generate answers full of repetitions or generic filler, which opens room for open-ended algorithm adjustments.

5 Next Steps

1. Complete DPO tuning experiments.
2. Run additional offline baselines (IPO and AOT).
3. Train and evaluate online RLHF methods.
4. Compare methods using held-out evaluation metrics.
5. Design and implement an open-ended investigation for the second part of the proposal.

A Technical details

Table 1 below shows the hyperparameters used in the current DPO implementation. The current version of the codebase is at https://github.com/Yehongqiu-lab/llm_rl, within the "final_project_llm_rl" folder.

β	learning rate	effective train batch size	epochs
5e-4	5e-6	32	2

Table 1: Hyper-parameters for the DPO algorithm.

B Engineering on the log-prob computation

We use the log of conditional probabilities throughout the LLM training, and the quantities are the backbone of computing any learning objective.

- Per-token log-probability: after aligning the output digits of a LM over one batch with the next-token targets, computing the cross-entropy, and then taking its negative value, we get the per-token log-prob: $\log p(x_t|x_{<t})$.

- How is $\pi_\theta(y|x)$ actually computed: From the per-token log-prob, apply a response mask, add up the unmasked per-token log-probs, and use the chain rule of probability to compute the log of the joint probability. But from the engineering perspective, We use the masked average of token log-probabilities to reduce sequence-length dependence and improve numerical stability.

C Additional Algorithm Details

C.1 Offline Methods

DPO:

The key contribution of the algorithm is to transform a loss function over reward functions into a loss function over policies. The change-of-variable approach avoids fitting an explicit and standalone reward model and optimizes directly over policies, bypassing the RL step completely. The policy objective(minimization) is:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log \sigma(\beta \Delta_\theta(x, y^+, y^-))]$$

IPO:

Implicit Preference Optimization uses the same reference-corrected margin $\Delta_\theta(x, y^+, y^-)$ as DPO, but instead of applying a logistic loss, it matches the margin to a target value:

$$\mathcal{L}_{\text{IPO}}(\pi_\theta; \pi_{\text{ref}}) = \mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} \left(\Delta_\theta(x, y^+, y^-) - \frac{1}{2\beta} \right)^2$$

The intuition for the method is: DPO keeps pushing to separate chosen and rejected responses through a logistic objective; while IPO treats the desired preference margin as a regression target. This often makes IPO feel more conservative than DPO: rather than endlessly increasing margins, it aims for a particular scale. IPO is therefore a useful comparison point because it changes the shape of the loss while keeping almost the same ingredients as DPO.

AOT:

Alignment via Optimal Transport(AOT) changes the granularity of comparison. Instead of only optimizing each chosen/rejected pair independently, it compares the distribution of chosen scores against the distribution of rejected scores across a minibatch. Each example (corresponding to a prompt x) produces a scalar reference-corrected reward:

$$r_\theta(x, y) = \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

The formula is precise up to a normalization constant with respect to x . AOT sorts the chosen rewards $r_\theta(x, y^+)$ and the rejected rewards $r_\theta(x, y^-)$ separately and then applies a DPO-style penalty to the sorted quantile gaps:

$$\mathcal{L}_{\text{AOT}} = \mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} \left(-\log \sigma(\beta(r_{(i)}^+ - r_{(i)}^-)) \right)$$

Here $r_{(i)}^+$ is the i -th sorted chosen reward and $r_{(i)}^-$ is the i -th sorted rejected reward. Intuitively, AOT encourages the entire chosen distribution to shift upward relative to the rejected distribution, not only each individual paired example.

C.2 Reward Model

The first phase in the online RLHF, is to model the human preferences based on the generated dataset $\{x_{(i)}, y_{(i)}^+, y_{(i)}^-\}_{i=1}^N$. Assume there is a latent reward model to be learnt: $r(x, y)$, and that the human preference can be modeled as a Bradley-Terry model:

$$p(y_1 \succ y_2 | x) = \frac{\exp(r(x, y_1))}{\exp(r(x, y_1)) + \exp(r(x, y_2))}$$

Parameterize the reward model with a neural network $r_\phi(x, y)$, the training loss objective is framed as a negative log likelihood:

$$\mathcal{L}_{\text{reward}}(r_\phi) = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log p(y^+ \succ y^- | x)]$$

In practice, the reward model is often initialized from the fixed base model π_{ref} with a linear layer on top of the final transformer layer that produces a single scalar prediction for the reward value. Prior works also normalize the learnt reward values to ensure a lower variance of the reward function.

C.3 Online Methods

Preliminary: KL regularization proxy

Online methods regularized the trainable policy toward the frozen reference policy. At one token position, define

$$\Delta(y_t) = \log \pi_{\text{ref}}(y_t | x, y_{<t}) - \log \pi_\theta(y_t | x, y_{<t}).$$

The KL divergence is $\mathbb{D}_{KL}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)] = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [-\sum_{t=1}^T \Delta(y_t)]$. So if using the sampled-token nonnegative KL proxy:

$$\hat{k}(a) \doteq \exp \Delta(a) - \Delta(a) - 1$$

The KL divergence proxy can be written as:

$$\hat{\mathbb{D}}_{KL}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)] = \sum_{t=1}^T \hat{k}(a)$$

Notes:

1. The expectation of the KL proxy under $y \sim \pi_\theta(\cdot|x)$ is the exact token-level KL:

$$\mathbb{D}_{KL}[\pi_\theta(y|x)||\pi_{\text{ref}}(y|x)] = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)}[\sum_{t=1}^T \hat{k}(a)]$$

2. By Taylor expansion, easy to verify that KL proxy is non-negative.

3. In practice, an engineering trick is to use the masked-average $\frac{1}{T} \sum_{t=1}^T \hat{k}(a)$ as a proxy.

GRPO:

The algorithm comes from the paper DeepSeekMath, in which they built a small-but-strong LLM model for math reasoning with meticulous data curation for pre-training and a new RLHF algorithm GRPO.

GRPO is derived from an actor-critic RL algorithm PPO(Policy Gradient Optimization).

The optimization objective for PPO is to maximize the smaller term in the unclipped and clipped terms:

$$\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(y|x)} \left\{ \frac{1}{T} \sum_{t=1}^T \min \left[\frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})} A_t, \text{clip} \left(\frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right] \right\}$$

In practice, there's also a KL penalty term:

$$\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(y|x)} \left(\frac{1}{T} \sum_{t=1}^T \min \left[\frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})} A_t, \text{clip} \left(\frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right] \right) - \beta \mathbb{D}_{KL}(\pi_\theta || \pi_{\text{ref}})$$

Notes:

1. The fraction $\frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ and the expectation over the old policy model $\pi_{\theta_{\text{old}}}$ is derived from the importance sampling of the off-policy learning(the vanilla PPO is on-policy, which means in every step of the gradient ascent, we need to sample new prompt-answer pairs from the current policy model).

2. If ignoring the KL term and the clipped gradient term, take the derivative of the objective function with respect to θ , we recover a formula very similar to the policy gradient of the off-policy PPO:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(y|x)} \left(\frac{1}{T} \sum_{t=1}^T \left(\frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})} \cdot \nabla_\theta \log \pi_\theta(y_t|x, y_{<t}) \cdot A_t \right) \right)$$

3. A_t is the advantage, and is computed by applying Generalized Advantage Estimation(GAE). GAE requires training a Value Model alongside the Policy Model.

4. Using a clipped surrogate objective is to stabilize the training process(an engineering trick) by preventing the policy from changing too much within one updating step. Note that the clipped gradient surrogate is different from the gradient norm clipping trick commonly used to prevent exploding gradients.

The GRPO bypasses the Value Model, and uses a Group-relative advantage in place of GAE. For one prompt x_i , suppose we sample G responses with rewards $r_{i,1}, \dots, r_{i,G}$. The group-relative advantages:

$$A_{i,j} = \frac{r_{i,j} - \mu_i}{\sigma_i + \epsilon}$$

with $\mu_i = \frac{1}{G} \sum_{j=1}^G r_{i,j}$,

$$\sigma_i = \sqrt{\frac{1}{G} (r_{i,j} - \mu_i)^2}.$$

DrGRPO:

DrGRPO makes two changes relative to GRPO: 1. it uses mean-centered group-relative advantages without dividing by the per-group standard deviation;

2. it removes the per-sequence length normalization inside the surrogate.

The motivation is that these changes alter how GRPO handles sequence length and reward scaling. In particular, removing the per-sequence normalization changes the length dependence of the update, which makes DrGRPO a useful comparison point when you are thinking about length bias in policy optimization.

GSPO:

Instead of clipping the token-level likelihood ratios as GRPO does, GSPO aggregates token log-ratios into a length-normalized single sequence-level ratio, and then applies the PPO-style clipping at the sequence level. This changes the optimization geometry: GSPO treats the response as one structured action rather than a bag of token-wise policy updates.